

GUÍA DE ESTUDIO · FUNDAMENTOS FRONTEND

JavaScript & TypeScript

Todo lo que necesitas para una entrevista fullstack (BBVA y similares): desde asignaciones e igualaciones hasta métodos, clases, declaraciones, dependencias y comandos. Con ejemplos reales del proyecto **Ventas-Frontend** (Angular 21 + TypeScript).

Declaraciones var / let / const

Asignaciones y mutación

Igualaciones == / ===

Métodos de arreglos y strings

Funciones y closures

Clases y herencia

Asincronía y Promesas

Módulos y dependencias (npm)

Comandos node / npm / ng

TypeScript completo

Tipos, interfaces y genéricos

Q&A de entrevista + Cheat-sheet

docs/Guia-Estudio-JavaScript-TypeScript.pdf · 2026-09-18 · OneDrive: Ventas-Frontend

Índice

01	JavaScript: qué es y dónde vive	02	Tipos de datos y valores
03	Declaraciones: var, let, const	04	Asignaciones y mutación
05	Igualaciones y comparaciones	06	Operadores
07	Strings y números (métodos)	08	Condicionales y bucles
09	Funciones y closures	10	Arreglos y sus métodos
11	Objetos, destructuring y JSON	12	Clases: métodos, herencia, getters
13	Asincronía: Promesas, async/await	14	Errores: throw, try / catch
15	Módulos y dependencias (package.json)	16	Comandos para ejecutar (node, npm, ng)
17	TypeScript: por qué y cómo compila	18	TS: tipos básicos
19	TS: interfaces, types y enums	20	TS: funciones tipadas y genéricos
21	TS: clases, modificadores y decoradores	22	TS: tsconfig, utility types, aserciones
23	TypeScript + Angular en este proyecto	24	Q&A típico de entrevista
25	Cheat-sheet final		

01 JavaScript: qué es y dónde vive BASE

JavaScript (JS) es el lenguaje de la web. Es **interpretado** (no se compila como Java/C#), **dinámico** (las variables cambian de tipo), **débilmente tipado** (los tipos se convierten entre sí) y **un solo hilo** (single-thread).

Dónde corre

Navegador	Motor V8 (Chrome), SpiderMonkey (Firefox). Accede al DOM, eventos y HTTP.
Node.js	El mismo lenguaje fuera del navegador: servidores, scripts, CLIs.
ECMAScript	Es el estándar. JS es su implementación. Ediciones ES6 (2015), ES2017, ES2022...
TypeScript	JS + tipos estáticos. Se compila a JS. Angular está escrito y se programa en TS.

Cómo se relaciona con TypeScript en Angular

.ts (TS)	→ <code>tsc</code> (compilador) →
.js (JS)	→ lo ejecutan V8 / Node o los navegadores.

Todo lo que haces en JS lo haces en TS, pero con tipos que te avisan de errores antes de ejecutar.

Frase clave en entrevista: "JS es un lenguaje dinámico e interpretado; TypeScript le añade tipos estáticos y se transpila a JS, dándome autocompletado y menos bugs en tiempo de desarrollo. Angular 21 corre sobre TypeScript estricto."

02 Tipos de datos y valores BASE

Primitivos (inmutables, por valor)

string	'hola', 'x', `plantilla`
number	enteros y decimales (solo <code>double</code>), <code>NaN</code> , <code>Infinity</code>
boolean	<code>true</code> / <code>false</code>
undefined	"no asignado todavía"
null	"ausencia intencional de valor"
symbol	identificador único
bigint	números muy grandes

Referencias (por dirección de memoria)

object	<code>{...}</code> colecciones de claves: valor
array	<code>[...]</code> , también es un objeto
function	también es un objeto "invocable"
Date, Map, Set...	objetos del runtime

El 70% de los bugs de primitivos vs referencias viene de aquí: copiar un objeto con `=` copia la referencia (no el contenido). Lo ves en la sección 04.

Truthy vs falsy (toda condición se convierte a booleano)

Falsy (solo hay 7)

<code>false</code>	el booleano
<code>0</code> , <code>-0</code> , <code>0n</code>	cero
<code>''</code> o <code>""</code>	string vacío
<code>null</code>	sin valor
<code>undefined</code>	sin asignar
<code>NaN</code>	no es un número

Truthy (todo lo demás)

<code>'0'</code>	string con el cero → truthy!
<code>[]</code>	arreglo vacío → truthy
<code>{}</code>	objeto vacío → truthy
<code>'false'</code>	el string "false" es truthy
<code>-1</code> , <code>3.14</code>	cualquier número ≠ 0

Cuidado: `if ([ ])` entra al if. Para comprobar vacío usa `arr.length`. En el proyecto se usa `filter(Boolean)` para descartar vacíos.

typeof

```
typeof 42;           // 'number'
typeof 'x';          // 'string'
typeof true;         // 'boolean'
typeof null;         // 'object' ← BUG histórico de JS ⚠️
typeof undefined;    // 'undefined'
typeof NaN;          // 'number' ← raro, pero es así
typeof [];           // 'object' ← para arreglos usa Array.isArray()
typeof fn;           // 'function'
```

03 Declaraciones: var, let, const

DECLARACIONES

El usuario pidió **declaraciones**: estas son las tres formas de declarar variables (la única pregunta obligada de entrevista).

Característica	var	let	const
Alcance (scope)	función (no bloque)	bloque { }	bloque { }
Hoisting	sí, se "sube" como <code>undefined</code>	sí, pero en zona muerta (TDZ)	igual que let
Reasignación	sí	sí	no
Volver a declararla	sí (puede)	no en el mismo scope	no
Recomendación	evítala	cuando cambia de valor	por defecto en un proyecto moderno

var vs let/const – scope y TDZ (Temporal Dead Zone)

```
// var "flota" y arranca como undefined
console.log(v); // undefined (no da error)
var v = 10;

// let/const arrancan en "zona muerta": usarlas antes = ReferenceError
console.log(x); // ❌ ReferenceError: Cannot access 'x'
let x = 5;

// const no se puede reasignar
const URL_API = '/api/products';
// URL_API = 'otra'; → TypeError: Assignment to constant variable

// El proyecto usa casi todo const (un componente, un servicio, un enum...)
const products = []; // ok: el ARREGLO cambia, la REFERENCIA no
```

Frase clave: "Uso `const` por defecto (inmutabilidad de referencia), `let` solo cuando la variable va a cambiar, y evito `var` porque ignora el bloque y causa bugs de scope."

Bonus: `const` NO congela el contenido. `const arr = []` admite `arr.push(1)`. Para "congelarlo" se usa `Object.freeze()` o `readonly` en TypeScript.

04 Asignaciones y mutación

ASIGNACIONES

Operadores de asignación

Operador	Equivale a	Ejemplo
<code>=</code>	asignar	<code>total = 10</code>
<code>+=</code> / <code>-=</code>	sumar / restar	<code>total += 5</code>
<code>*=</code> / <code>/=</code> / <code>%=</code>	mult / div / módulo	<code>total *= 3</code>
<code>**=</code>	potencia	<code>total **= 2</code>
<code>??=</code>	asigna solo si es <code>null/undefined</code>	<code>cfg.timeout ??= 3000</code>
<code> =</code> / <code>&&=</code>	asigna según falsy / truthy	<code>cfg.enabled = true</code>

asignación.js – por valor vs por referencia (el punto más importante)

```
// PRIMITIVOS → se copia el VALOR
let a = 10;
let b = a;
b = 99;
console.log(a); // 10 (a no cambió)

// OBJETOS/ARREGLOS → se copia la REFERENCIA (misma caja en memoria)
const p = { price: 10 };
const q = p;
q.price = 99;
console.log(p.price); // 99 ✳ ¡mutó el original!

// Copia real (shallow) para no mutar
const copia = { ...p, price: 25 }; // {...} + spread
const copiaArr = [...items];      // clon de arreglo

// Así lo hace el proyecto (cobro-service.ts): nunca muta el estado directo
const current = [...this.cart$.value]; // copia la lista
current.push(producto);                // muta la copia
this.cart$.next(current);              // publica el nuevo estado
```

Para entrevista de BBVA: te pueden preguntar “¿mutar varios objetos con la misma referencia es un problema?”. Respuesta: sí, produce **efectos colaterales**. Se soluciona con copias (spread, `Object.assign`), inmutabilidad y en Angular con señales/BehaviorSubject que se actualizan con copias nuevas.

05 Igualaciones y comparaciones **IGUALACIONES**

Operador	Significado	Resultado de <code>'1' == 1</code>
<code>==</code>	igualdad laxa : convierte tipos (coerción)	true (convierte)
<code>===</code>	igualdad estricta : valor Y tipo sin convertir	false
<code>!=</code> / <code>!==</code>	diferente laxa / estricta	—
<code><</code> <code>></code> <code><=</code> <code>>=</code>	relacionales (comparan por valor)	—
<code>Object.is(a,b)</code>	más preciso que <code>===</code>	—

igualaciones.js – los 5 casos trampa

```
// 1) Regla de oro del entrevistador: SIEMPRE usa === (y !==)
'0' == 0; // true (coerción: string → number)
'0' === 0; // false
false == 0; // true 🤔 (evita ==)
false === 0; // false

// 2) null vs undefined
null == undefined; // true
null === undefined; // false

// 3) NaN nunca es igual a sí mismo
NaN === NaN; // false → usa Number.isNaN(x)
Object.is(NaN, NaN); // true (una excepción)

// 4) Los OBJETOS se comparan por referencia, NO por contenido
{ id: 1 } === { id: 1 }; // false – son 2 cajas distintas
const x = { id: 1 };
const y = x; // misma referencia
x === y; // true

// En el proyecto se compara por id (nunca el objeto entero)
this.products = this.products.filter(p => p.id !== id);
```

Frase clave: “Siempre comparo con `===`. Para objetos comparo su `id` (y en test se usa `toEqual` que hace comparación profunda).” `Object.is` lo mencionas solo si te retan con `NaN` o `-0`.

06 Operadores

BASE

Aritméticos y lógicos

<code>+</code> <code>-</code> <code>*</code> <code>/</code>	aritmética; <code>+</code> también concatena strings
<code>%</code>	módulo (resto)
<code>**</code>	potencia
<code>++</code> <code>--</code>	incremento / decremento
<code>&&</code> <code>/</code> <code> </code>	Y / O (con cortocircuito)
<code>!</code>	negación
<code>??</code>	nullish coalescing: valor si no es null/undefined
<code>?.</code>	encadenamiento opcional (no rompe)
<code>? a : b</code>	ternario (if en línea)

Cortocircuito y modernos

```
const r = a && b; // si a es falsy → a; si no → b
const r2 = a || b; // si a es truthy → a; si no → b
const r3 = a ?? b; // si a es null/undefined → b
const n = obj?.precio?.total; // seguro, nunca peta
const r4 = cond ? 'SI' : 'NO';
```

Diferencia clave `||` vs `??`: `0 || 'a' → 'a'` (0 es falsy). `0 ?? 'a' → 0` (0 no es null/undefined).

Uso real en Ventas-Frontend (sale-facade.ts / reportes.ts)

```
const errorMessage = err.error?.message || 'Pago rechazado';
// 1) err.error?.message → si existiera, su mensaje (opcional)
// 2) || 'Pago rechazado' → si no, texto por defecto (cortocircuito)

this.cashSummary?.openingAmount ?? 0
// si viene el dato lo usa; si es null/undefined usa 0 (??)

const items = savedItems ?? this.cobro.cart$.value;
```

07 Strings y números (métodos)

MÉTODOS

Métodos de string (devuelven nuevo string)

<code>length</code>	número de caracteres
<code>toUpperCase()</code> / <code>toLowerCase()</code>	mayúsculas / minúsculas
<code>includes('x')</code>	¿contiene? → el proyecto lo usa en el interceptor
<code>startsWith</code> / <code>endsWith</code>	¿empieza/termina con?
<code>indexOf('x')</code>	posición o -1
<code>slice(1, 3)</code> / <code>substring</code>	extraer parte
<code>split(',')</code>	string → arreglo
<code>replace('a','b')</code> / <code>replaceAll</code>	reemplazar
<code>trim()</code>	quita espacios de los bordes
<code>padStart(2, '0')</code>	rellena a la izquierda

Números

<code>toFixed(2)</code>	2 decimales (devuelve string)
<code>toLocaleString('es-MX')</code>	formato con separadores: 1,234.5
<code>Number('42')</code> / <code>Number('a')</code>	→ 42 / NaN
<code>parseInt('2.9')</code>	→ 2 (entero)
<code>Math.round</code> / <code>floor</code> / <code>ceil</code>	redondeos
<code>Math.random()</code>	0–1 (aleatorio)
<code>Math.min</code> / <code>Math.max</code>	mínimo / máximo

strings.js + template literals (backticks `\${...}`)

```
'hola'.toUpperCase();           // 'HOLA'
'ABCDE'.includes('BC');          // true
'a,b,c'.split(',');              // ['a','b','c']
['a','b'].join('-');             // 'a-b' ← lo inverso
' venta '.trim();                // 'venta'
```

```
// Template Literal: interpolar sin '+':
const ticket = `Total: ${total.toFixed(2)}`;
const url = `${environment.api}/products`;
```

```
// Uso real en el proyecto (reportes.ts → PDF):
`Caja #${this.cashSummary?.cashId ?? '-'}`
```

08 Condicionales y bucles

BASE

Condicionales

```
if (stock > 0) {
  // venta posible
} else if (stock === 0) {
  // agotado
} else {
  // negativo: problema
}

switch (metodo) {
  case 'CASH': comision = 0; break;
  case 'DEBIT': comision = 0.03; break;
  default: comision = 0.05;
}
```

Truthy/falsy: `if (usuario)` basta; no hace falta `!= null` en JS plano (TS strict sí lo pide a veces).

Bucles

```
for (let i = 0; i < 5; i++) { ... } // clásico
for (const item of items) { ... } // valores de un arreglo ✓
for (const key in obj) { ... } // claves de un objeto
items.forEach(it => ...); // método, no devuelve nada
while (cond) { ... }
do { ... } while (cond);
```

Preferencia: en el proyecto casi no se usan `for`: se usa `.map` / `.filter` / `.reduce` / `.forEach`, que son más legibles.
`break` / `continue` funcionan en todos.

Condición real (auth-interceptor.ts)

```
const isAuthEndpoint =
  req.url.includes('/auth/login') ||
  req.url.includes('/auth/forgot-password') ||
  req.url.includes('/auth/reset-password');

if (!isAuthEndpoint) {
  safeRemoveItem('token');
  safeRemoveItem('user');
  router.navigate(['/']);
}
```

09 Funciones y closures

FUNCIONES

funciones.js — las 3 formas + parámetros + rest

```
// 1) Declaración (hoisting: se puede llamar antes de declararla)
function sumar(a, b = 0) { // b tiene valor por defecto
  return a + b;
}

// 2) Expresión como const
const restar = function (a, b) { return a - b; };

// 3) Arrow function (la más usada en el proyecto)
const doble = (n) => n * 2; // return implícito si es 1 línea
const suma = (a, b) => a + b;

// Parámetros REST ... → junta el resto en un arreglo
function totalTodo(...nums) {
  return nums.reduce((acc, n) => acc + n, 0);
}
totalTodo(1, 2, 3); // 6
```

Arrow vs function normal: la gran diferencia es `this`

`function()` — `this` = quien la llama

```
const boton = {
  label: 'OK',
  normal: function() {
    return this.label; // 'OK'
  }
};
boton.normal(); // this = boton
```

`() =>` — `this` = contexto donde se define

```
function Componente() {
  this.lista = [];
  this.load = () => this.lista; // this = Componente
  // si fuera "function()", this sería window/undefined
}
```

En Angular: las flechas hacen que `this` sea el componente (evita perder el contexto en callbacks de RxJS).

Closure (clausura)

closure.js — la función recuerda su entorno

```
function crearContador() {
  let n = 0; // vive "atrapada" en el closure
  return () => ++n; // la flecha recuerda a 'n'
}
const c1 = crearContador();
c1(); c1(); c1(); // 1, 2, 3 — cada una tiene su propio 'n'
const c2 = crearContador();
c2(); // 1 (independiente)
```

Frase clave: “Una closure es una función que conserva acceso a las variables de su ámbito aunque este ya terminó. En el proyecto se ve en los callbacks de `subscribe()`, que capturan el estado del componente cuando llega la respuesta HTTP.”

Extras de entrevista: `call()` / `apply()` / `bind()` controlan el `this` de una función. **IIFE:** `(function(){...})()` se ejecuta al instante.

10 Arreglos y sus métodos

MÉTODOS

Los métodos de arreglos son probablemente **lo más preguntado en pruebas técnicas**. Dos familias:

Mutadores (cambian el arreglo)

<code>push(x)</code>	agrega al final → devuelve tamaño
<code>pop()</code>	quita el último
<code>shift()</code> / <code>unshift(x)</code>	quita / agrega al inicio
<code>splice(i, n)</code>	corta n desde i (también inserta)
<code>sort()</code>	ordena. ⚠ por default como STRING
<code>reverse()</code>	invierte

Inmutables (devuelven copia — las del proyecto)

<code>map(fn)</code>	transforma cada elemento
<code>filter(fn)</code>	deja los que cumplen
<code>reduce(fn, ini)</code>	reduce a un valor (suma, máximo...)
<code>forEach(fn)</code>	recorre sin devolver nada
<code>find(fn)</code> / <code>findIndex</code>	primer elemento que cumple / su índice
<code>some(fn)</code> / <code>every(fn)</code>	¿alguno / todos cumplen?
<code>includes(x)</code>	¿existe el valor?
<code>slice(i, j)</code>	copia desde i hasta j
<code>concat([...])</code>	une arreglos
<code>flat()</code>	aplana

arreglos.js — los 4 imprescindibles (map / filter / reduce / find)

```
const numeros = [1, 2, 3, 4];

numeros.map(n => n * 2);           // [2, 4, 6, 8] → transformar
numeros.filter(n => n % 2 === 0);   // [2, 4]      → filtrar
numeros.reduce((suma, n) => suma + n, 0); // 10 → acumular
numeros.find(n => n > 2);           // 3 (el primero)
[10, 2].sort((a, b) => a - b);     // [2, 10] orden numérico
```

Uso real en Ventas-Frontend (salehistory.ts, compras.ts, sale-facade.ts)

```
// filter para buscar y comparar por id
result = result.filter(sale => sale.paymentMethod === this.selectedMethod);
this.users = this.users.filter(u => u.id !== id);

// reduce para el TOTAL (con ?? para sumar solo lo que existe)
const total = this.filteredSales.reduce((sum, s) => sum + (s.total ?? 0), 0);

// map + filter para transformar series de una API
this.categories = [...new Set(
  products.map(p => p.categoryName).filter(Boolean)
)];

// map para "devolver" un objeto nuevo
items: items.map(item => ({ ... }));
```

Frase clave: “Prefiero métodos funcionales e inmutables (map/filter/reduce) porque no mutan el estado; en Angular eso destruye la detección de cambios de señales y causa bugs difíciles de rastrear.”

11 Objetos, destructuring y JSON OBJETOS

objetos.js

```
const product = { id: 1, name: 'Coca', price: 18.5 };

product.price;           // acceso con punto      → 18.5
product['name'];         // con corchete           → 'Coca'
product.stock = 3;       // agrega / modifica clave
delete product.price;    // borra la clave

// Recorrer claves del objeto
Object.keys(p);          // ['id', 'name', 'price']
Object.values(p);        // [1, 'Coca', 18.5]
Object.entries(p);       // [['id', 1], ['name', 'Coca'], ...]
```

destructuring (extraer en variables) y spread

```
// Objetos
const { id, price } = product;           // dos variables con esos valores
const { name, ...resto } = product;      // name + el resto en 'resto'

// Arreglos
const [a, b] = [10, 20, 30]; // a=10, b=20
const [primero, ...rest] = arr;

// Copiar + sobrescribir (composición)
const combo = { ...product, price: 25 };
```

JSON (formato de intercambio, no es JS)

```
const texto = JSON.stringify({ id: 1, name: 'x' });
// '{"id":1,"name":"x"}' - claves siempre entre comillas dobles

const obj = JSON.parse('{ "a":1 }'); // {a: 1} / ⚠ puede lanzar, usa try/catch

// El backend (Spring Boot) y el frontend se pasan JSON por HTTP.
```

En TS esto mapea 1:1 con interfaces: el JSON que envía el backend se escribe como `export interface ProductForm { ... }` y Angular ya sabe qué forma tendrá. Lo ves en la sección 19.

12 Clases: métodos, herencia, getters CLASES

En ES6 las clases son **azúcar sintáctica sobre prototipos**: nada de lo clásico de Java, pero la sintaxis te resulta familiar.

clases.js

```
class Product {
  taxRate = 1.16;           // campo con valor inicial

  constructor(name, price) { // se ejecuta con new
    this.name = name;
    this.price = price;
  }

  get grandTotal() {        // GETTER: se lee SIN paréntesis
    return this.price * this.taxRate;
  }

  set discount(v) { this.price = this.price - v; } // SETTER

  static buildEmpty() {     // método estático: sin instancia
    return new Product('', 0);
  }
}

class DigitalProduct extends Product { // HERENCIA
  constructor(name, price, url) {
    super(name, price);           // llama al constructor padre
    this.url = url;
  }
  get grandTotal() { return this.price; } // override del getter
}

const lata = new Product('Lata', 18.5);
lata.grandTotal;           // 21.46 (getter, sin "()")
lata instanceof Product;   // true

const ebook = new DigitalProduct('PDF', 99, 'url');
ebook instanceof Product;  // true - recorre la cadena de herencia
ebook.grandTotal;          // 99 (override del padre)
Product.buildEmpty();      // método estático
```

Cómo se ve una clase real en el proyecto (product-service.ts)

```
@Injectable({ providedIn: 'root' })
export class ProductService {
  private apiUrl = `${environment.api}/products`;

  constructor(private http: HttpClient) {} // inyección de dependencia

  getProducts(): Observable<ProductForm[]> {
    return this.http.get<ProductForm[]>(this.apiUrl);
  }

  deactivateProduct(id: number): Observable<ProductForm> {
    return this.http.patch<ProductForm>(
      `${this.apiUrl}/${id}/deactivate`, {}
    );
  }
}
```

Frase clave: “En JS las clases son azúcar sobre prototipos. Angular las usa como servicios (`@Injectable`) y componentes (`@Component`), con inyección de dependencias en el constructor.” En TS se les suma `public/private/protected/readonly` (sección 21).

13 Asincronía: Promesas, async/await ASINCRONÍA

JS es de **un solo hilo**: la asincronía no bloquea. El **event loop** saca tareas de la cola. Microtareas (Promesas) corren antes que macrotareas (setTimeout, I/O).

promesas.js

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve('ok'), 1000); // éxito
  // reject(new Error('fallé')) // rechazo
});

p.then(r => console.log(r)) // 'ok'
  .catch(err => console.error(err)) // error por separado
  .finally(() => limpiar()); // siempre

// Combinar:
Promise.all([a, b]); // espera TODAS; cae si alguna falla
Promise.allSettled([a, b]); // espera TODAS; reporta cada una (nunca rechaza)
Promise.race([a, b]); // La primera en resolver
```

async / await (azúcar sobre promesas – más legible)

```
async function cargar() {
  try {
    const res = await fetch('/api/sales');
    const data = await res.json();
    return data;
  } catch (err) {
    console.error('No se pudo cargar', err);
  } finally {
    loader(false); // se ejecuta siempre
  }
}

// await solo se usa dentro de funciones async
setTimeout(() => console.log('macro tarea'), 0); // va DESPUÉS
Promise.resolve().then(() => console.log('micro tarea')); // va ANTES
```

Opción importante: el navegador ya trae `fetch()` para HTTP. En Angular NO se usa: se usa `HttpClient` (RxJS) que devuelve **Observables**, no Promesas. RxJS es la versión potente de asincronía del proyecto: `pipe()`, `map()`, `catchError()`, `subscribe()`. En entrevista di: “conozco Promesas y async/await; en Angular los servicios usan Observables de RxJS, que permiten cancelación y operadores.”

RxJS real en auth-interceptor.ts

```
return next(req).pipe(
  catchError((err) => {
    if (err instanceof HttpResponse && err.status === 401) {
      router.navigate(['/']);
    }
    return throwError(() => err); // vuelve a lanzar el error
  })
);
```

14 Errores: throw, try / catch **ERRORES**

errores.js

```
throw new Error('Stock insuficiente'); // lanzar a propósito

try {
  const cfg = JSON.parse(texto);
} catch (err) {
  // err es un objeto (Error, TypeError, SyntaxError, ReferenceError...)
  console.warn(err instanceof SyntaxError ? 'JSON inválido' : err);
} finally {
  cerrarLoader(); // SIEMPRE llega aquí (aunque haya return)
}

// Los más comunes
TypeError // operación sobre tipo incorrecto
ReferenceError // variable no declarada
SyntaxError // código mal escrito
RangeError // número fuera de rango
HttpResponse // en Angular: respuesta HTTP con error
```

Aplicado al proyecto: los componentes muestran `alert(err.error?.message || '...')` cuando un servicio falla; el `AuthInterceptor` convierte un 401 en "cerrar sesión y navegar a /". El backend devuelve excepciones con campo `message`, por eso el frontend lee `err.error?.message`.

15 Módulos y dependencias (package.json)

DEPENDENCIAS

Import / export (ES Modules)

modulos.ts (así funciona en toda la app)

```
// archivo: core/enums/paymentMethod.ts
export enum PaymentMethod {
  CASH = 'CASH', DEBIT = 'DEBIT', CREDIT = 'CREDIT',
}

// otro archivo lo importa por nombre:
import { PaymentMethod } from '../../core/enums/paymentMethod';
import { Component } from '@angular/core';
import { HttpClient } from '@angular/common/http';

// export por defecto (1 por archivo) se importa sin llaves:
export default function init() {}
import init from './init';
```

package.json: dónde vive cada dependencia

package.json de este proyecto (resumido)

```
{
  "name": "demo-ira-car",
  "scripts": {
    "start": "ng serve",
    "build": "ng build",
    "test": "ng test"
  },
  "dependencies": {
    "@angular/core": "^21.0.0",
    "rxjs": "^7.8.1",      // PRODUCCIÓN: necesita subir con la app
    "chart.js": "^4.5.1",
    "jspdf": "^4.2.1"
  },
  "devDependencies": {
    "vitest": "^3.0.0",    // SOLO desarrollo/testing
    "typescript": "~5.9.0"
  }
}
```

Concepto	Qué es	Cómo se usa
package.json	manifiesto: nombre, scripts, dependencias	se crea con <code>npm init</code>
dependencies	rango semver, p. ej. <code>^21.0.0</code>	<code>npm install x</code>
devDependencies	solo build/test (vitest, typescript)	<code>npm install -D x</code>
node_modules/	dependencias instaladas (no va a git)	aparece con <code>npm install</code>
package-lock.json	versiones EXACTAS congeladas	<code>npm ci</code> la respeta al pie

Semver MAYOR.MENOR.PATCH : `^7.8.1` permite `7.x.x` (solo menor). `~7.8.1` permite `7.8.x` (solo patch). `npm install` crea package-lock para reproducibilidad; `npm ci` instala exacto (usado en CI/Docker).

16 Comandos para ejecutar

COMANDOS

Node y npm (JavaScript)

```
node app.js           # ejecuta un script JS
node --watch app.js   # ambiente de desarrollo con
                      # recarga
node -v               # versión de Node
npm -v                # versión de npm

npm init -y           # crea package.json rápido
npm install rxjs       # (alias npm i) usa cd
                      # dependencias
npm install -D vitest  # devDependency
npm install -g @angular/cli # global (CLI de Angular)
npm uninstall lodash   # quitar
npm update             # actualiza según semver
npm audit              # vulnerabilidades
npm ci                 # limpio y EXACTO desde lock
npm run build          # ejecuta scripts.build
npx ng serve           # corre paquetes sin instalarlos
```

Angular CLI (este proyecto)

```
npm start              # = ng serve → :4200
npx ng serve           # servidor de desarrollo
npx ng build           # build de producción
npx ng test            # Vitest, una vez y sale
npx ng test --watch=false
npx ng generate component features/xyz
npx ng generate service core/service/xyz
npx ng version         # versión de Angular
npx ng lint            # estilo de código
```

En entrevista: "El proyecto lo levanto con `npm start` (ng serve en el puerto 4200), compilo con `npx ng build` y corro los tests con `npx ng test --watch=false`. npm resuelve dependencias por package.json + package-lock; `npm ci` garantiza el mismo árbol en CI."

17 TypeScript: por qué y cómo compila

TYPESCRIPT

Pregunta	Respuesta
¿Qué es?	JavaScript con tipos estáticos (superset: todo JS válido es TS válido). Creado por Microsoft.
¿Cómo funciona?	El compilador <code>tsc</code> transpila <code>.ts</code> → <code>.js</code> . Los tipos desaparecen en runtime (son solo para el editor y el build).
¿Por qué Angular lo usa?	Autocompletado, errores al escribir (no al ejecutar), refactors seguros, contratos con el backend y los decoradores necesitan tipos.
¿Strict?	Angular 21 usa <code>"strict": true</code> : errores más estrictos (null checks, funciones, switch).

app.ts → tsc → app.js

```
// TypeScript: tipos explícitos o inferidos
const total: number = 42;
let nombre: string = 'Ana';

function suma(a: number, b: number): number {
  return a + b;
}
// ✖ suma('x', 2) → error de compilación ANTES de ejecutar

// El navegador solo entiende JS: tsc borra los tipos
// const total = 42; → así queda en el .js final
```

Cómo se compila en este proyecto (angular.json)

```
"fileReplacements": [{
  // producción usa environment-prod.ts
  "replace": "environments/environment.ts",
  "with": "environments/environment-prod.ts"
}]
// ng build → (tsc/extract) → dist/Demo-IraCar/browser + SSR
// ng test → Vitest + @angular-builders → entiende .ts directo
```

Frase clave: "TS es JS con tipos que se borran al compilar; me da autocompletado y descarta bugs antes del runtime. Angular lo exige y lo usa estricto."

18 TS: tipos básicos

TYPESCRIPT

tipos.ts

```
let nombre: string = 'Ana';
let stock: number = 10;
let activo: boolean = true;

// any: evítalo (apaga la protección)
let cualquiera: any = 'cualquier cosa';

// unknown: seguro, hay que "acotar" antes (narrowing)
let dato: unknown = '{"a":1}';
if (typeof dato === 'string') { dato.toUpperCase(); }

let nulo: null = null;
let indef: undefined = undefined;

function saludar(n: string): void { console.log(n); } // sin retorno

// Inferencia: TS deduce el tipo por ti
const apellido = 'Pérez'; // es string, no hace falta anotarlo

// Arreglos y tuples
const ids: number[] = [1, 2, 3];
const gas: Array<number> = [1, 2];
const pareja: [string, number] = ['CASH', 3]; // tupla

// never: nunca retorna / siempre lanza
function fatal(mensaje: string): never {
  throw new Error(mensaje);
}

// void: no devuelve nada
function log(x: string): void { console.log(x); }
```

Tipo	Para qué	Regla de oro
any	desactivar tipos	EMBARCAR: nunca usar en código nuevo
unknown	valor desconocido	acotar con typeof/instanceof antes de usar
void	funciones sin retorno	por defecto en callbacks sin return
never	funciones que tiran excepción	saber que existe (preguntan con frecuencia)
boolean number string	clásicos	prefiere la inferencia a anotar todo

19 TS: interfaces, types y enums

TIPADO / DECLARACIONES

Interface (contrato de forma de un objeto)

core/interfaces/product/product.ts — REAL

```
export interface ProductForm {
  id?: number;           // ? = opcional (puede faltar)
  name: string;          // obligatorio
  price: number;
  stock: number;
  img?: string;
  categoryId: number;
  hasHistory?: boolean;  // el backend decide si se puede eliminar
  readonly sku: string;  // readonly: no se reasigna
  createdAt?: Date;
}
// Uso en runtime: La interface "desaparece"; solo ordena el código.
```

Type (alias / uniones / intersecciones)

tipos-avanzados.ts

```
type ID = string | number;           // unión
type MetodoPago = 'CASH' | 'DEBIT' | 'CREDIT'; // literales exactos

type CarritoItem = CobroItem & { extra?: number }; // intersección: todas las props
// un objeto debe cumplir LAS DOS partes

// interface vs type:
// interface → se puede extender y re-abrir (declaration merging)
// type      → no se puede re-declarar (pero soporta uniones/mapeado)
interface Animal { nombre: string }
interface Perro extends Animal { ladra(): void }
```

Enum (constantes con nombre)

core/enums/paymentMethod.ts — REAL

```
export enum PaymentMethod {
  CASH = 'CASH',
  DEBIT = 'DEBIT',
  CREDIT = 'CREDIT',
}
// se usa como literal: sale.paymentMethod === PaymentMethod.CASH
// y en el HTML del POS para poner el badge correcto.
```

Enum vs unión: el enum da autocompletado y reusabilidad; la unión (type) es más ligera. Muchos proyectos modernos prefieren `const object + union`. En Angular, los enums se usan igual: son objetos de runtime.

20 TS: funciones tipadas y genéricos

FUNCIONES TS

funciones-ts.ts

```
// Parámetros y retorno tipados + parámetro opcional
function calcular(precio: number, qty = 1, iva?: number): number {
  const base = precio * qty;
  return iva ? base + base * iva : base;
}

// Tipo de una función como variable (callback)
const aplicar: (fn: (n: number) => number, v: number) => number =
  (fn, v) => fn(v);
aplicar((n) => n * 2, 5); // 10

// GENÉRICOS: captura el tipo y lo propaga
function primero<T>(lista: T[]): T | undefined {
  return lista[0];
}
primero([1, 2, 3]); // T = number → devuelve number | undefined
primero(['a', 'b']); // T = string

// Todo en Angular/RxJS es genérico:
Observable<ProductForm[]> // la lista que el backend devuelve
BehaviorSubject<CobroItem[]> // estado reactivo del carrito
Promise<Response> // fetch tipado
```

Frase clave: "Los genéricos conectan el tipo de entrada con el de salida ($T[] \rightarrow T$), dándome tipado sin perder flexibilidad. RxJS está lleno de ellos: `Observable<T>`, `BehaviorSubject<T>`, `HttpClient.get<T>()`."

21 TS: clases, modificadores y decoradores

CLASES TS

clases-ts.ts – lo que JS NO tiene

```
class Cuenta {
  public   face: string;      // público: cualquiera (default)
  private saldo: number;     // solo dentro de la clase
  protected historial: string[]; // clase y subclases
  readonly id: number;       // inmutable tras el constructor

  constructor(id: number) {
    this.id = id;
    this.saldo = 0;
    this.face = 'directo';
  }

  depositar(monto: number): void { this.saldo += monto; }
  total(): number { return this.saldo; }
}

// #campo: privacidad de LENGUAJE (ES2022) – no solo de compilador
class Caja {
  #interna = 100;
  abrir() { return this.#interna; }
}

// implements: La clase DEBE cumplir la interface
interface Listable { getLabel(): string }
class Fila implements Listable {
  getLabel() { return 'fila'; }
}
```

Decoradores en Angular (sí, TS los hace posibles)

componente + servicio del proyecto

```
@Component({
  selector: 'app-productos',
  standalone: true,           // Angular 21: sin NgModules
  templateUrl: './productos.html',
  styleUrls: ['./productos.css']
})
export class Productos implements OnInit {
  products: ProductForm[] = [];
  ngOnInit(): void { this.loadProducts(); }
}

@Injectable({ providedIn: 'root' }) // singleton en la app
export class ProductService {
  constructor(private http: HttpClient) {} // DI del HttpClient
}
```

Frase clave: “Un decorador es una función que envuelve a la clase (o a un método/campo). Angular los usa para marcar componentes, servicios, pipes y directivas; `providedIn: 'root'` hace del servicio un singleton inyectable.” `@Injectable`, `@Component`, `@Input`, `@Output`, `@Pipe` son decoradores de Angular.

22 TS: tsconfig, utility types, aserciones

HERRAMIENTAS TS

tsconfig.json: la configuración del compilador

tsconfig.json (Angular 21) – resumido

```
{
  "compilerOptions": {
    "strict": true,      // null-checks, tipos implícitos prohibidos...
    "target": "ES2022",
    "module": "preserve",
    "experimentalDecorators": true,  // necesario para @Component
    "useDefineForClassFields": true,
    "noUnusedLocals": true,
    "strictNullChecks": true,        // null/undefined importan en tipos
    "forceConsistentCasingInFileNames": true
  }
}
```

Utility types (los que te preguntan siempre)

utility-types.ts

```
type SoloNombre = Pick<ProductForm, 'name' | 'price'>;    // elige campos
type SinId       = Omit<ProductForm, 'id' | 'createdAt'>;  // quita campos
type Parcial     = Partial<ProductForm>;                  // todo opcional ?
type Completo    = Required<ProductForm>;                 // todo obligatorio
type Congelado   = Readonly<ProductForm>;                 // todo readonly
type Diccionario = Record<string, number>;                // mapa clave:valor
```

Aserciones y narrowing

aserciones.ts

```
const precio = valor as number;      // casting directo (cuidado)
const metodo = "CASH" as const;      // literal exacto 'CASH'
const nombre  = datoIngresado!;      // "non-null": juro que no es null ⚠️

// Narrowing: acortar tipos antes de usarlos
function imprimir(dato: string | string[]) {
  if (Array.isArray(dato)) { dato.forEach(console.log); }
  else { console.log(dato); }
}
```

Regla: prefiere narrow (typeof, instanceof, Array.isArray) y utility types; usa `as` y `!` solo como último recurso (rompen las verificaciones).

23 TypeScript + Angular en este proyecto

EJEMPLO REAL

Recorrido de una sola venta para ver TS trabajando de verdad:

1. El contrato llega del backend → interface

core/interfaces/cobro/cobro.ts

```
export interface CobroItem {
  product: ProductShow;
  quantity: number;
  unitPrice: number;
  subtotal: number;
}
```

2. El estado reactivo → clase + genérico + BehaviorSubject

core/service/cobro-service/cobro-service.ts

```
@Injectable({ providedIn: 'root' })
export class CobroService {
  cart$ = new BehaviorSubject<CobroItem[]>([]); // Lista tipada

  add(product: ProductShow) {
    const current = [...this.cart$.value];
    const existing = current.find(item => item.product.id === product.id);
    if (existing) {
      if (existing.quantity >= product.stock) {
        alert('Stock insuficiente');
        return;
      }
      existing.quantity++;
      existing.subtotal = existing.quantity * existing.unitPrice;
    } else {
      current.push({ product, quantity: 1, unitPrice: product.price, subtotal: product.price });
    }
    this.cart$.next(current); // solo ahora se publica
  }

  getTotalLocal(): Observable<number> {
    return this.cart$.pipe(map(items =>
      items.reduce((sum, item) => sum + item.subtotal, 0)));
  }
}
```

3. HTTP tipado + funciones provider (interceptor)

El 401 se ataja en auth-interceptor.ts

```
export const AuthInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router); // DI en función
  const token = safeGetItem('token'); // helper TS
  if (token) {
    req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  }
  return next(req).pipe(
    catchError((err) => {
      if (err instanceof HttpErrorResponse && err.status === 401) {
        safeRemoveItem('token');
        safeRemoveItem('user');
        router.navigate(['/']);
      }
      return throwError(() => err);
    })
  );
};
```

Lo que demuestras con este recorrido: (1) interfaces para DTOs, (2) genéricos de RxJS, (3) decoradores y DI, (4) tipado en HTTP, (5) narrowing con `instanceof`, (6) manejo de errores con alzas de tipos. Es exactamente el perfil que buscan en BBVA fullstack.

24 Q&A típico de entrevista

ENTREVISTA

¿Diferencia entre let, const y var?

var ignora el bloque (scope de función) y sufre hoisting como undefined; **let** respeta el bloque y tiene TDZ; **const** es como let pero no se puede reasignar la referencia. Hoy uso const por defecto, let cuando cambia, var casi nunca.

¿== vs ===?

`==` convierte tipos (`'1' == 1` → true); `===` exige mismo valor Y tipo. Uso siempre `===` (y `!==`). Curiosidades: `null == undefined` → true; `NaN === NaN` → false; objetos se comparan por referencia.

¿Qué es un closure?

Una función que conserva acceso al scope donde nació aunque esa función ya terminó. Ej: un contador que mantiene su contador interno.

¿Qué es hoisting?

JS "sube" las declaraciones de var y function al inicio de su scope. var queda como undefined; let/const quedan en zona muerta y dan ReferenceError si se usan antes.

¿Diferencia entre map y forEach?

forEach no devuelve nada; map devuelve un arreglo nuevo con el resultado de cada llamada. map sirve para transformar, forEach solo para recorrer.

¿Shallow vs deep copy?

Shallow copia el nivel superficial (`{...obj}`); los objetos anidados siguen compartidos. Deep copia todo (structuredClone, JSON.parse/stringify, librerías).

¿Promise vs async/await?

Son lo mismo: async/await es azúcar sobre promesas. Las promesas encadenan con .then/.catch; async/await las lee como código síncrono con try/catch.

¿Cómo funciona el event loop?

JS es de un solo hilo. Las operaciones asíncronas se delegan y sus callbacks vuelven por colas: primero **microtareas** (Promesas) y luego **macrotareas** (setTimeout, I/O). El loop los saca cuando el stack está vacío.

¿null vs undefined?

undefined = aún no se asigna un valor; null = ausencia intencional de valor. Ambos son falsy. `??` solo reacciona a estos dos.

¿Qué es truthy/falsy?

Todo valor se convierte a booleano en un if. Falsy: false, 0, "", null, undefined, NaN. Todo lo demás (incluidos [] y {}) es truthy.

¿Cómo ordenar números con sort?

`arr.sort((a,b) => a - b)`. Por defecto sort compara como string: `[10,2].sort()` da [10,2] porque "10" < "2".

¿Qué es TypeScript y por qué lo usa Angular?

JS con tipos estáticos que se borran al compilar. Da autocompletado, errores tempranos y refactors seguros. Angular 21 es TS estricto.

¿any vs unknown?

any desactiva el chequeo; unknown la mantiene y obliga a acotar (narrow) antes de usar. Prefiero unknown.

¿interface vs type?

Ambos describen la forma de un objeto. interface se puede extender y re-abrir; type soporta uniones (|), intersecciones (&) y mapeados. Unión/literales solo con type; extender contratos, interface.

¿Qué es un genérico?

Un tipo "parámetro" `<T>` que conecta la entrada con la salida. En RxJS: `Observable<T>` , `BehaviorSubject<T>` ; en Angular: `HttpClient.get<T>()` .

¿Qué es un Observable vs una Promise?

Promise resuelve UNA vez y no se cancela; Observable emite N valores en el tiempo, es perezoso, se asigna con `subscribe()` y se puede cancelar con `unsubscribe()` . Angular's `HttpClient` devuelve Observables.

¿Qué hace un decorador?

Es una función que envuelve/metadatoa a una clase o miembro. En Angular: `@Component` , `@Injectable` , `@Input` , `@Output` , `@Pipe` . El compilador usa esos metadatos para generar la app.

¿Qué contiene package-lock.json?

Las versiones exactas instaladas (transitivas incluidas). Garantiza builds reproducibles. `npm ci` instala según ese archivo; `npm install` lo actualiza.

¿Qué es strict mode y qué cambia?

tsconfig con un conjunto de reglas estrictas: no null sin anotar, sin parámetros implícitos any, switch exhaustivo... Saca más errores en tiempo de desarrollo. Angular 21 lo activa.

¿Cómo se actualiza un paquete?

`npm update` (respetar semver), `npm install pkg@version` , o `npm audit fix` para vulnerabilidades. Luego correr `ng build` y los tests.

25 Cheat-sheet final

RESUMEN

Declaraciones

```
const x = 1;
let y = 2;    // se reasigna
var z;        // evítalo
// const no congela: solo la referencia
```

Igualdad

```
a === b;  a !== b;  // SIEMPRE
a == b;    // nunca en código nuevo
Object.is(a, b);   // NaN y -0
```

Arreglos

```
map    // transforma
filter // filtra
reduce // acumula
find   // primer match
some / every // alguno / todos
includes // existe
slice  // corta copia
splice // corta mutando
```

Strings

```
'abc'.toUpperCase()
'abc'.includes('b')
'a,b'.split(',')    // → ['a','b']
['a','b'].join('-') // → 'a-b'
`Total: ${n.toFixed(2)}`
```

Asincronía

```
fetch(URL).then(r => r.json())
  .catch(console.error)

async function f() {
  try { const r = await fetch(URL); }
  catch (e) { /* 4xx/5xx */ }
  finally { /* siempre */ }
}
```

TypeScript

```
const n: number = 1;
interface X { id?: number; }
type ID = number | string;
function g<T>(x: T): T { return x; }
// TS borra tipos al compilar
```

Angular (TS en acción)

```
@Component({ standalone: true })
export class X implements OnInit {
  ngOnInit() {}
}

@Injectable({ providedIn: 'root' })
export class Service {
  constructor(private http: HttpClient) {}
}

http.get<ProductForm[]>(url).subscribe(...)
```

Comandos

```
node app.js
node --watch app.js
npm i pkg          // install
npm i -D pkg       // dev
npm ci             // desde lock
npm run build
npm start           // ng serve :4200
npx ng test --watch=false
```

Objetivo de la guía: si dominas declaraciones, asignaciones vs mutación, igualaciones estrictas, métodos de arreglos/strings, clases, asincronía (Observables incluidos), módulos + npm y lo que TS añade (tipos, interfaces, enums, genéricos, decoradores, strict), tienes la base fullstack que BBVA valida en las pruebas técnicas. ¡Éxito!